

## Author

Vedant Sharma

25F2009080

25f2009080@ds.study.iitm.ac.in

I am a diploma student at the IITM BS Degree program with an interest in the domains of system design and the backend logics behind core cs functionalities.

## AI/LLM usage

21%

8% Bootstrap

10% Authentication

3% Validation

## Description

The project involves developing a web-based Placement Portal that connects students, companies, and administrators through a centralized platform. It supports job posting and applications, student/company management, interview and placement workflow.

I used AI/LLM to help with the project structure, improve design with Bootstrap, introduce token based authentication, validation and Redis caching.

## Technologies used

Flask - Core backend framework for developing REST APIs and server-side application logic.

SQLAlchemy - Database abstraction and management of application entities/relationships.

SQLite - Lightweight relational database for storing users, students, companies, jobs data.

Flask-JWT-Extended - JWT based auth, managing user sessions.

Flask-Bcrypt - Secure password hashing and verification.

Flask-CORS - Communication between Vue.js and Flask

Flask-Caching - API Response caching

Redis - Message broker for celery.

Celery - Handles asynchronous background tasks.

Vue.js - Frontend Javascript framework.

Vue Router - Handles navigation.

Axios - Performs HTTP requests between Vue.js and Flask APIs.

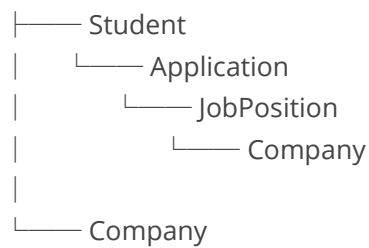
Bootstrap - Responsive UI components.

Mailtrap SMTP - Sandboxing email features and testing.

# DB Schema Design

Relationships:

User



| Table       | Important Columns                                                                                                                             | Constraints / Purpose                                                                                     |
|-------------|-----------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| User        | id, username, password, email, role                                                                                                           | id is Primary Key; username is Unique and Not Null; role identifies Student, Company or Admin             |
| Student     | id, student_id, name, department, contact, cgpa, skills, education, experience, resume, is_active, user_id                                    | id is Primary Key; student_id is Unique; user_id is Foreign Key to User                                   |
| Company     | id, company_id, name, description, industry, hr_contact, website, approved, is_active, user_id                                                | id is Primary Key; company_id is Unique; user_id is Foreign Key to User; approval controls company access |
| JobPosition | id, title, description, salary, location, deadline, required_skills, experience, min_cgpa, application_limit, status, is_approved, company_id | id is Primary Key; company_id is Foreign Key to Company; job approval/status controls availability        |
| Application | id, student_id, job_id, status, remarks, applied_at, updated_at, interview_datetime, interview_mode, interview_location, interview_notes      | id is Primary Key; student_id and job_id are Foreign Keys; connects students with job positions           |
| Placement   | Placement/application-related placement details such as position, salary, joining date and offer letter                                       | Stores final placement information associated with a successful application                               |

Constraints:

**Primary Keys** uniquely identify each database record.

**Foreign Keys** connect related entities such as Student → User and Application → Student/Job.

**Unique constraints** prevent duplicate usernames and company/student identifiers.

**Not Null constraints** ensure essential information is always available.

**Default values** are used for fields such as account activity, approval and job/application statuses.

**Boolean fields** such as approved and is\_active simplify account and approval management.

The database was designed using a normalized relational structure so that information is stored only where it logically belongs. For example, company information is stored separately from job information, while applications connect students and jobs through foreign keys.

Separating the entities makes the system easier to maintain and allows one company to have multiple job positions and one student to apply for multiple positions. Foreign-key relationships and constraints also help prevent invalid or inconsistent records.

## API Design

| API Area            | Purpose                                                                                   |
|---------------------|-------------------------------------------------------------------------------------------|
| Authentication APIs | Registration, login, logout and user authentication                                       |
| Student APIs        | Profile management, resume handling, job search, applications and application history     |
| Company APIs        | Company profile, job creation/management, applicants, interviews and placement processing |
| Admin APIs          | Student/company management, job approval, application management and dashboard statistics |
| Export APIs         | Asynchronous generation and downloading of application-history CSV reports                |
| Interview APIs      | Interview scheduling and related student information                                      |
| Caching             | Redis caching for frequently accessed API responses                                       |

The API design separates frontend presentation from backend business logic, allowing the Vue.js interface and Flask backend to communicate independently and making the system easier to maintain and extend.

# Architecture and Features

## Architecture

The Placement Portal follows a three-layer architecture consisting of a Vue.js frontend, Flask backend, and SQLite database. The frontend is organized into Vue views and components, with Vue Router handling navigation and Axios communicating with the backend REST APIs. The Flask backend is organized into route modules for authentication, students, companies and administrators, while models define the database entities and relationships. Services handle reusable functionality such as email operations, while Celery tasks handle asynchronous operations such as interview reminders and report generation. Redis is used for caching and as the Celery message broker. The SQLite database stores the persistent application data.

## Features Implemented

The system provides separate functionality for Students, Companies and Administrators.

- **Student features:** registration and authentication, profile management, CGPA and skills, resume upload/viewing, placement-drive search, job applications, application-status tracking, interview details, offer letters and application-history CSV export.
- **Company features:** company registration and approval, profile management, job creation and management, applicant viewing, application status updates, interview scheduling, and placement/offer processing.
- **Admin features:** dashboard statistics, company approval/rejection, student and company management, job and application management, and account blocking/deactivation.
- **Additional features:** JWT-based authentication, role-based access control, Redis API caching, Celery background processing, automated interview reminder emails through SMTP/Mailtrap, and asynchronous CSV report generation.

## Video

<https://drive.google.com/file/d/1hIK23eootpKCC9utEv9EqalvpStH8ajO/view?usp=sharing>